Python Dictionary - Beginner Notes

1. Create a Dictionary

What is a Dictionary?
A dictionary is an **unordered collection** of data stored as **key:value pairs**. Keys must be unique.

Syntax:

Using curly braces

```
student = {"name": "Alice", "age": 20, "grade": "A"}
print(student) # Output: {'name': 'Alice', 'age': 20, 'grade': 'A'}
```

Using dict() function

```
student2 = dict(name="Bob", age=22, grade="B")
print(student2) # Output: {'name': 'Bob', 'age': 22, 'grade': 'B'}
```

Empty dictionary

```
empty_dict = {}
print(empty_dict) # Output: {}
```

Dictionary with different data types

```
mixed = {
"name": "John",
"age": 25,
"marks": [85, 90, 95],
"is_student": True
}
print(mixed)
```

Key Points:

- Keys must be unique and immutable (strings, numbers, tuples)
- Values can be any data type
- Dictionaries are mutable (can be changed)

2. Accessing Items

You can access values using keys.

student = {"name": "Alice", "age": 20, "city": "Mumbai"}

Access using square brackets

print(student["name"]) # Output: Alice

Access using get() method (safer - no error if key doesn't exist)

print(student.get("age")) # Output: 20
print(student.get("phone")) # Output: None

Access all keys

print(student.keys()) # Output: dict_keys(['name', 'age', 'city'])

Access all values

print(student.values()) # Output: dict_values(['Alice', 20, 'Mumbai'])

Access all key-value pairs

print(student.items()) # Output: dict_items([('name', 'Alice'), ('age', 20), ('city', 'Mumbai')])

Check if key exists

print("name" in student) # Output: True
print("phone" in student) # Output: False

3. Adding Key-Value Pair

student = {"name": "Alice", "age": 20}
print(student) # Output: {'name': 'Alice', 'age': 20}

Add single key-value pair

student["city"] = "Mumbai"
print(student) # Output: {'name': 'Alice', 'age': 20, 'city': 'Mumbai'}

Add another key-value pair

student["grade"] = "A"
print(student) # Output: {'name': 'Alice', 'age': 20, 'city': 'Mumbai', 'grade': 'A'}

Add multiple key-value pairs using update()

student.update({"phone": "9876543210", "email": "alice@example.com"})
print(student)

4. Editing Key-Value Pair

student = {"name": "Alice", "age": 20, "grade": "B"}
print(student) # Output: {'name': 'Alice', 'age': 20, 'grade': 'B'}

Edit existing value

student["age"] = 21
print(student) # Output: {'name': 'Alice', 'age': 21, 'grade': 'B'}

Edit using update()

student.update({"grade": "A"})
print(student) # Output: {'name': 'Alice', 'age': 21, 'grade': 'A'}

Edit multiple values

student.update({"age": 22, "grade": "A+"})
print(student) # Output: {'name': 'Alice', 'age': 22, 'grade': 'A+'}

5. Dictionary Operations

a) Delete Operations

student = {"name": "Alice", "age": 20, "city": "Mumbai", "grade": "A"}

Remove specific key using pop()

removed_value = student.pop("city")
print(removed_value) # Output: Mumbai
print(student) # Output: {'name': 'Alice', 'age': 20, 'grade': 'A'}

Remove last inserted item using popitem()

```
last_item = student.popitem()
print(last_item) # Output: ('grade', 'A')
print(student) # Output: {'name': 'Alice', 'age': 20}
```

Delete specific key using del

```
student["age"] = 20
student["city"] = "Mumbai"
del student["city"]
print(student) # Output: {'name': 'Alice', 'age': 20}
```

Clear all items

```
student.clear()
print(student) # Output: {}
```

b) Copy Dictionary

```
original = {"a": 1, "b": 2, "c": 3}
```

Shallow copy using copy()

```
copied = original.copy()
print(copied) # Output: {'a': 1, 'b': 2, 'c': 3}
```

Copy using dict()

```
copied2 = dict(original)
print(copied2) # Output: {'a': 1, 'b': 2, 'c': 3}
```

c) Merge Dictionaries

```
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}
```

Merge using update()

```
dict1.update(dict2)
print(dict1) # Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Merge using | operator (Python 3.9+)

```
dict3 = {"x": 10, "y": 20}
dict4 = {"z": 30}
merged = dict3 | dict4
print(merged) # Output: {'x': 10, 'y': 20, 'z': 30}
```

d) Loop Through Dictionary

```
student = {"name": "Alice", "age": 20, "city": "Mumbai"}
```

Loop through keys

```
for key in student:
print(key)
```

Output: name, age, city

Loop through values

```
for value in student.values():
print(value)
```

Output: Alice, 20, Mumbai

Loop through key-value pairs

```
for key, value in student.items():
print(f"{key}: {value}")
```

Output: name: Alice, age: 20, city: Mumbai

6. Dictionary Functions

Function	Description
len(dict)	Returns number of key-value pairs
str(dict)	Converts dictionary to string
type(dict)	Returns type of variable

Table 1: Common Dictionary Functions

Dictionary Methods:

Method	Description
keys()	Returns all keys
values()	Returns all values
items()	Returns all key-value pairs
get(key)	Returns value for key (no error)
pop(key)	Removes key and returns value
popitem()	Removes last inserted item
update()	Updates dictionary with key-value pairs
clear()	Removes all items
copy()	Returns shallow copy
fromkeys()	Creates dict from keys with same value
setdefault()	Returns value or sets default

Table 2: Common Dictionary Methods

Examples:

student = {"name": "Alice", "age": 20, "city": "Mumbai"}

Length

print(len(student)) # Output: 3

Get with default value

print(student.get("phone", "Not Available")) # Output: Not Available

Setdefault (adds key if not exists)

student.setdefault("grade", "A")
print(student) # Output: {'name': 'Alice', 'age': 20, 'city': 'Mumbai', 'grade': 'A'}

Fromkeys

keys = ["a", "b", "c"]
new_dict = dict.fromkeys(keys, 0)
print(new_dict) # Output: {'a': 0, 'b': 0, 'c': 0}

7. Dictionary Comprehension

Dictionary comprehension creates a new dictionary in a single line.

Syntax: `{key_expression: value_expression for item in iterable if condition}`

Example 1: Square of numbers

```
squares = {x: x**2 for x in range(1, 6)}
print(squares) # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Example 2: Filter even numbers

```
numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
even_squares = {x: x**2 for x in numbers if x % 2 == 0}
print(even_squares) # Output: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

Example 3: Convert to uppercase

```
fruits = ["apple", "banana", "cherry"]
fruit_dict = {fruit: fruit.upper() for fruit in fruits}
print(fruit_dict) # Output: {'apple': 'APPLE', 'banana': 'BANANA', 'cherry': 'CHERRY'}
```

Example 4: Create dictionary with length of words

```
words = ["Python", "Java", "C++"]
word_length = {word: len(word) for word in words}
print(word_length) # Output: {'Python': 6, 'Java': 4, 'C++': 3}
```

Example 5: Conditional dictionary comprehension

```
marks = {"Alice": 85, "Bob": 60, "Charlie": 90, "David": 45}
passed = {name: mark for name, mark in marks.items() if mark >= 50}
print(passed) # Output: {'Alice': 85, 'Bob': 60, 'Charlie': 90}
```

8. zip()

The `zip()` function combines multiple iterables (lists, tuples) into tuples.

Syntax: `zip(iterable1, iterable2, ...)`

Example 1: Zip two lists

```
names = ["Alice", "Bob", "Charlie"]
ages = [20, 22, 21]
result = zip(names, ages)
print(list(result)) # Output: [('Alice', 20), ('Bob', 22), ('Charlie', 21)]
```

Example 2: Create dictionary from two lists using zip()

```
keys = ["name", "age", "city"]
values = ["Alice", 20, "Mumbai"]
student = dict(zip(keys, values))
print(student) # Output: {'name': 'Alice', 'age': 20, 'city': 'Mumbai'}
```

Example 3: Zip three lists

```
subjects = ["Math", "Science", "English"]
marks = [85, 90, 88]
grades = ["A", "A+", "A"]
result = list(zip(subjects, marks, grades))
print(result) # Output: [('Math', 85, 'A'), ('Science', 90, 'A+'), ('English', 88, 'A')]
```

Example 4: Zip with different lengths (stops at shortest)

```
list1 = [1, 2, 3, 4, 5]
list2 = ['a', 'b', 'c']
result = list(zip(list1, list2))
print(result) # Output: [(1, 'a'), (2, 'b'), (3, 'c')]
```

Example 5: Unzip using zip with *

```
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
numbers, letters = zip(*pairs)
print(numbers) # Output: (1, 2, 3)
print(letters) # Output: ('a', 'b', 'c')
```

Example 6: Dictionary comprehension with zip()

```
keys = ["id", "name", "age"]
values = [101, "Alice", 25]
dictionary = {k: v for k, v in zip(keys, values)}
print(dictionary) # Output: {'id': 101, 'name': 'Alice', 'age': 25}
```

9. Nested Comprehensions

Nested comprehensions create dictionaries with nested structures (dictionary inside dictionary).

a) Nested Dictionary Comprehension

Example 1: Create nested dictionary

```
students = ["Alice", "Bob", "Charlie"]
subjects = ["Math", "Science"]
```

Nested dictionary with default marks 0

```
result = {student: {subject: 0 for subject in subjects} for student in students}
print(result)
```

Output: {'Alice': {'Math': 0, 'Science': 0}, 'Bob': {'Math': 0, 'Science': 0}, 'Charlie': {'Math': 0, 'Science': 0}}

Example 2: Multiplication table

```
table = {i: {j: i*j for j in range(1, 4)} for i in range(1, 4)}
print(table)
```

Output: {1: {1: 1, 2: 2, 3: 3}, 2: {1: 2, 2: 4, 3: 6}, 3: {1: 3, 2: 6, 3: 9}}

Example 3: Nested dictionary with conditions

```
numbers = [1, 2, 3, 4, 5]
result = {x: {"square": x2, "cube": x3} for x in numbers if x % 2 == 0}
print(result)
```

Output: {2: {'square': 4, 'cube': 8}, 4: {'square': 16, 'cube': 64}}

b) Nested Loop in Dictionary Comprehension

Example 4: Flatten nested structure

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat_dict = {f"item_{i}": value for row in matrix for i, value in enumerate(row)}
print(flat_dict)
```

Output: {'item_0': 1, 'item_1': 2, 'item_2': 3, 'item_0': 4, ...}

Example 5: Create dictionary from nested lists

```
students = [["Alice", 85], ["Bob", 90], ["Charlie", 88]]
marks_dict = {name: mark for name, mark in students}
print(marks_dict)
```

Output: {'Alice': 85, 'Bob': 90, 'Charlie': 88}

c) Complex Nested Comprehension

Example 6: Nested dictionary with multiple conditions

```
result = {
x: {y: x+y for y in range(1, 4) if y != x}
for x in range(1, 4)
}
print(result)
```

Output: {1: {2: 3, 3: 4}, 2: {1: 3, 3: 5}, 3: {1: 4, 2: 5}}

Example 7: Real-world example - Student grades by subject

```
students = ["Alice", "Bob"]
subjects = {"Math": 85, "Science": 90}
grades = {student: {subject: mark for subject, mark in subjects.items()} for student in students}
print(grades)
```

Output: {'Alice': {'Math': 85, 'Science': 90}, 'Bob': {'Math': 85, 'Science': 90}}

Key Points:

- Outer loop creates main dictionary keys
- Inner loop creates nested dictionary values
- Can combine with conditions for filtering
- Useful for creating complex data structures quickly

Quick Reference

Operation	Code
Create dict	d = {"a": 1, "b": 2}
Access value	d["a"] or d.get("a")
Add item	d["c"] = 3
Edit item	d["a"] = 10
Remove item	d.pop("a")
Loop items	for k, v in d.items()
Comprehension	{x: x**2 for x in range(5)}
Zip to dict	dict(zip(keys, values))
Nested dict	{x: {y: x*y for y in [1,2]} for x in [1,2]}

Table 3: Dictionary Operations Quick Reference

Concept	Purpose
Dictionary	Store key-value pairs
keys()	Get all keys
values()	Get all values
items()	Get all key-value pairs
get()	Safe access (no error)
Comprehension	Create dict in one line
zip()	Combine iterables
Nested dict	Dictionary inside dictionary

Table 4: Dictionary Concepts Summary